Build · Ship · Run · Scale

Một cuốn sổ duy nhất...
Đầy đủ mọi thứ về Docker!

〈 Cẩm nang 〉

Docker

Toàn Tập Đầy Đủ

Từ Cơ Bản đến Nâng Cao

- 📦 Docker Basics
- ▶ Docker CLI
- 📄 Dockerfile
- 🗂 Images & Containers
- 🔗 Networking
- 🗄 Volumes & Storage
- 🧩 Docker Compose
- 🛡 Security Best Practices
- ☁ Registries (Docker Hub, ECR)
- 🚀 CI/CD + Production Setup
- ⚙ Troubleshooting
- 📊 Interview Questions

~ Container giúp DevOps nhanh hơn, đơn giản hơn và tốt hơn! ♥



- ✓ Simple
- ✓ Practical
- ✓ Real-World
- ✓ Production Ready

✓ Học Docker ♥
Từng Bước Một!

Từ Con Số 0 🚀
Đến Chuyên Gia



- Commands
- Examples
- Best Practices
- Real Use Cases
- Interview Q&A

Cùng Chinh Phục Docker! 🔥

Lời mở đầu từ Mentor

Đọc trang này trước. Nó quyết định bạn học Docker trong 6 tuần hay 6 tháng.

Chào bạn. Từ hôm nay mình là mentor Docker của bạn. Cuốn sổ phía sau có 20 trang ghi chú rất đẹp, nhưng đọc hết 20 trang không làm bạn biết Docker — cũng như đọc sách bơi không làm bạn biết bơi. Docker là kỹ năng **tay**: bạn biết nó khi bạn đã gõ lệnh, làm hỏng container, đọc log và tự sửa được.

Vì vậy mình đã sắp lại các trang theo **thứ tự học** (không phải thứ tự chủ đề), chia thành 7 phase. Mỗi phase có một bài lab bắt buộc và một tiêu chí "xong" rõ ràng. Chưa đạt tiêu chí thì chưa qua phase sau.

Bạn sẽ đạt được gì?

- ✔ Đóng gói một app React/Vite thành image nhỏ gọn, build nhanh nhờ cache.
- ✔ Dựng cả stack **Frontend + Spring Boot API + MySQL** bằng đúng một lệnh `docker compose up -d`.
- ✔ Viết Dockerfile đạt chuẩn production: non-root, multi-stage, không lộ secret, đã scan lỗ hổng.
- ✔ Debug được container lỗi trong 5 phút theo một quy trình cố định.
- ✔ Trả lời tự tin 10 câu phỏng vấn Docker phổ biến nhất.

Cách dùng cuốn sổ này

① Đọc 1 trang — gõ ngay
Mỗi trang ghi chú: đọc 10 phút, rồi mở terminal gõ lại **mọi** lệnh trên trang. Không copy-paste.

② Phá có chủ đích
Sau mỗi bài, cố tình làm hỏng 1 thứ (sai port, xoá volume, sai tag...) và đọc kỹ thông báo lỗi.

③ Ghi "sổ lỗi"
Mỗi lỗi gặp phải ghi 3 dòng: triệu chứng → nguyên nhân → cách sửa. Đây là tài sản quý nhất khi đi làm.

④ Giải thích lại
Cuối mỗi phase, viết 5 dòng giải thích cho một người chưa biết Docker. Không giải thích được = chưa hiểu.

💡 **Nhịp học đề xuất**

60–90 phút/ngày, 5 ngày/tuần, khoảng 6 tuần. 20% thời gian đọc, 80% thời gian gõ lệnh. Đều đặn quan trọng hơn cường độ: 60 phút mỗi ngày tốt hơn hẳn 6 tiếng dồn vào Chủ nhật.

— Mentor của bạn. Hỏi mình bất cứ lúc nào bạn bị kẹt quá 30 phút.

Lộ Trình 7 Phase

Mỗi phase: đọc các trang ghi chú → làm lab → đạt tiêu chí "xong" rồi mới đi tiếp.

1 Hiểu Docker 2–3 buổi

📖 Docker là gì? · Kiến trúc Docker · Cài đặt & kiến thức cơ bản — tr. 7–9

Lab: Cài Docker (Windows: Docker Desktop + WSL2; Ubuntu: docker-ce). Chạy `hello-world`, rồi `docker run -d -p 8080:80 nginx` và mở localhost:8080.

Xong khi: Kể lại được từng bước khi gõ `docker run nginx`: client → daemon → registry → containerd/runc.

2 Container & Image 3–4 buổi

📖 Docker Containers · Docker Images — tr. 10–11

Lab: Chạy nginx, `exec` vào sửa index.html, stop/start (dữ liệu còn), rm rồi run lại (dữ liệu mất). So size `nginx:latest` với `nginx:alpine`, xem `docker history`.

Xong khi: Phân biệt image vs container, stop vs rm, tag vs image ID mà không cần nhìn tài liệu.

3 Dockerfile & Build 4–5 buổi

📖 Dockerfile — nền tảng · Build & tối ưu image · Dockerfile — thực hành production — tr. 12–14

Lab: Dockerize app **React + Vite**: multi-stage (node build → nginx:alpine). Ghi size trước/sau tối ưu. Sửa 1 file trong src, rebuild và xem layer nào dùng cache.

Xong khi: Image frontend dưới ~50 MB; giải thích được vì sao `COPY package*.json` phải đứng trước `COPY . .`

4 Network & Storage 3 buổi

📖 Docker Networking · Volumes & Storage — tr. 15–16

Lab: Tạo network riêng, chạy MySQL + Adminer, kết nối bằng tên container. Xoá container MySQL, chạy lại với cùng named volume → dữ liệu vẫn còn.

Xong khi: Giải thích được vì sao `localhost` bên trong container không phải máy của bạn.

5 Compose & cấu hình 4 buổi

📖 Docker Compose · Environment & Secrets · Compose production setup — tr. 17–19

Lab: Viết `compose.yaml` 3 service: React (nginx) + **Spring Boot API** + MySQL, có healthcheck, `depends_on: condition: service_healthy`, file `.env`.

Xong khi: Clone repo trên máy sạch, chạy `docker compose up -d` là cả stack hoạt động.

6 Vận hành production 4–5 buổi

📖 Docker Security · Quản lý tài nguyên · Docker Registry · Logging & Troubleshooting — tr. 20–23

Lab: Chuyển image sang non-root, thử `--read-only`. Scan bằng Trivy/Docker Scout, giảm CVE bằng cách đổi base image. Push Docker Hub với tag semver + git SHA. Đặt `--memory=64m` và cố tình gây OOM để thấy exit 137. Bật log rotation.

Xong khi: Dockerfile của bạn qua được checklist bảo mật ở trang Security.

7 Tổng ôn & phỏng vấn 3 buổi

📖 Production + Interview — tr. 24

Lab: Tự trả lời thành tiếng 10 câu phỏng vấn ở trang cuối. Viết GitHub Actions: push code → build → push image lên registry.

Xong khi: Hoàn thành Dự án tốt nghiệp (trang 25).

6 Hình Dung Cốt Lõi

Nắm chắc 6 điều này thì mọi lệnh Docker đều trở nên "hiển nhiên". Quay lại trang này mỗi khi thấy rối.

1. Image = class, Container = object

Image là bản thiết kế chỉ đọc. Container là một instance đang chạy. Từ 1 image tạo được 100 container, giống `new` 100 object từ 1 class.

2. Container là process, không phải VM

Container là một tiến trình Linux bị "nhốt": **namespaces** giới hạn nó thấy gì, **cgroups** giới hạn nó dùng bao nhiêu. Process chính (PID 1) thoát → container dừng.

3. Layer giống commit Git

Mỗi bước RUN/COPY/ADD tạo một layer xếp chồng. Cache đi từ trên xuống: một layer đổi thì **mọi layer phía dưới** phải build lại. Thứ ít đổi đặt lên đầu.

4. Mỗi container có localhost riêng

`localhost` trong container là chính nó. Container gọi nhau bằng **tên service** trên cùng network; máy ngoài gọi vào qua `-p host:container`.

5. Filesystem container là tạm

Ghi vào container = ghi lên giấy nháp. Còn khi stop, mất khi `rm`. Thứ gì cần sống sót (DB, upload) phải nằm trong **volume**.

6. Một image, mọi môi trường

Build một lần, chạy ở dev/staging/prod. Khác biệt giữa môi trường đi vào từ **bên ngoài**: env, file config, secrets — không bao giờ build image riêng cho từng môi trường.

Phản xạ debug — thuộc lòng thứ tự này

`docker ps -a` → container còn sống không? exit code bao nhiêu? (**137** = bị kill/OOM, **1** = app lỗi, **127** = không tìm thấy lệnh)

`docker logs --tail 100 <c>` → app nói gì trước khi chết?

`docker inspect <c>` → env, mounts, network, port mapping có đúng như bạn nghĩ?

`docker exec -it <c> sh` → vào trong kiểm tra file, gọi thử service khác bằng tên.

🎯 Với nền tảng frontend của bạn

Docker sẽ thay đổi cách bạn làm việc ở 3 chỗ: **(1)** không còn "máy em chạy được" khi bàn giao app React; **(2)** dựng backend + DB local bằng 1 lệnh để tự test API mà không phải cài MySQL/Java vào máy; **(3)** hiểu pipeline CI/CD mà team DevOps đang chạy. Các lab ở trang trước được chọn theo đúng 3 mục tiêu này.

Bút Đỏ Của Mentor

Bộ ghi chú rất tốt, nhưng có vài chỗ đã cũ hoặc dễ hiểu sai. Ghi nhớ các đính chính này khi đọc các trang tương ứng.

1. **[Volumes & Storage]** ~~"Dữ liệu mất khi container dừng."~~ Thực tế: dữ liệu ghi trong container **vẫn còn khi stop**, chỉ mất khi `docker rm`. (Riêng tmpfs thì mất khi dừng — câu đó đúng.)
2. **[Docker Images]** ~~"Mỗi chỉ thị tạo một layer."~~ Chỉ **RUN, COPY, ADD** tạo layer filesystem. ENV, EXPOSE, CMD, WORKDIR... chỉ ghi metadata, không làm image nặng thêm.
3. **[Dockerfile production]** ~~`docker scan`~~ đã ngừng hỗ trợ. Dùng `docker scout cves myapp:1.0` hoặc `trivy image myapp:1.0`.
4. **[Dockerfile production]** `COPY` rồi `RUN chown -R` tạo thêm 1 layer chứa bản sao toàn bộ file → image nặng gấp đôi phần đó. Viết gọn: `COPY --chown=appuser:appgroup target/app.jar app.jar`.
5. **[Compose (nhiều trang)]** ~~`version: "3.9"`~~ — Docker Compose v2 bỏ qua trường này và báo là obsolete. Bắt đầu file trực tiếp bằng `services:`.
6. **[Build & tối ưu]** ~~`export DOCKER_BUILDKIT=1`~~ không cần nữa: BuildKit đã là builder mặc định từ Docker Engine 23.0 và trong Docker Desktop.
7. **[Environment & Secrets]** Secrets "được lưu đã mã hoá" chỉ đúng với **Swarm**. Với `docker compose` thường, secret dạng `file:` chỉ được mount vào `/run/secrets/` — không mã hoá, nhưng vẫn tốt hơn env vì không lộ qua `docker inspect`. `external: true` cần Swarm.
8. **[Cài đặt Docker]** `apt install docker.io` là gói do Ubuntu đóng gói, thường cũ hơn. Tài liệu chính thức khuyên cài `docker-ce` từ repo của Docker (kèm plugin compose và buildx). Trên Windows, Docker Desktop chạy qua **WSL2**.
9. **[Quản lý tài nguyên]** `--cpu-shares` không "đặt trước" CPU: nó là trọng số tương đối và chỉ có tác dụng khi các container **tranh chấp** CPU.
10. **[Cleanup / Tài nguyên]** ⚠ `docker system prune -a --volumes -f` xoá cả volume không gắn container — tức là có thể **xoá luôn dữ liệu database**. Luôn chạy `docker system df -v` xem trước.

> 💡 **Bài học lớn hơn**
>
> Docker thay đổi nhanh. Khi một lệnh trong tài liệu (kể cả cuốn này) báo lỗi hoặc cảnh báo, đó không phải lỗi của bạn — hãy kiểm tra **docs.docker.com**. Thói quen đối chiếu tài liệu chính thức là kỹ năng của người làm nghề thật.

Mục Lục

Đã sắp theo thứ tự học. Bảng tra cứu lệnh nằm ở cuối sổ để dùng hằng ngày.

★ **Ghi chú Mentor**

Lời mở đầu & cách dùng sổ 2
Lộ trình 7 phase 3
6 hình dung cốt lõi + phản xạ debug 4
Bút đỏ — đính chính & cập nhật 5

1 **Phase 1 — Hiểu Docker**

Docker là gì? 7
Kiến trúc Docker 8
Cài đặt & kiến thức cơ bản 9

2 **Phase 2 — Container & Image**

Docker Containers 10
Docker Images 11

3 **Phase 3 — Dockerfile & Build**

Dockerfile — nền tảng 12
Build & tối ưu image 13
Dockerfile — thực hành production 14

4 **Phase 4 — Network & Storage**

Docker Networking 15
Volumes & Storage 16

5 **Phase 5 — Compose & cấu hình**

Docker Compose 17
Environment & Secrets 18
Compose production setup 19

6 **Phase 6 — Vận hành production**

Docker Security 20
Quản lý tài nguyên 21
Docker Registry 22
Logging & Troubleshooting 23

7 **Phase 7 — Tổng ôn & phỏng vấn**

Production + Interview 24

★ **Kết thúc**

Dự án tốt nghiệp & tự đánh giá 25
Phụ lục — Bảng tra cứu lệnh (Cheat Sheet) 26

Ngày: 08/08/2025

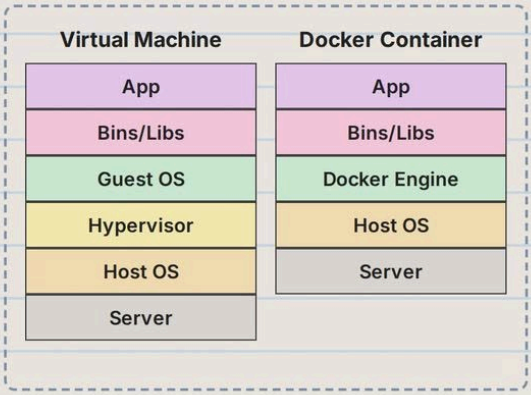
Docker Là Gì?

→ **Định nghĩa đơn giản:**

Docker là một nền tảng **mã nguồn mở** giúp chúng ta **build, ship** và **run** ứng dụng bên trong các **container nhẹ, dễ di chuyển**. Nó đóng gói ứng dụng + các dependency + runtime thành một đơn vị duy nhất gọi là **container**.

→ Tại Sao Dùng Docker? (Lợi Ích Chính)

- ✓ **Consistency (Nhất quán):** Hoạt động giống nhau trên mọi máy (Dev/QA/Prod)
- ✓ **Portability (Di động):** Build một lần, chạy ở bất kỳ đâu
- ✓ **Isolation (Cô lập):** Mỗi ứng dụng chạy trong container riêng
- ✓ **Lightweight (Nhẹ):** Dùng ít tài nguyên hơn so với VM
- ✓ **Faster Deployment (Triển khai nhanh):** Khởi động container trong vài giây



→ Docker Hoạt Động Như Thế Nào?



→ Các Lệnh Docker Thông Dụng:

```
docker --version          # Kiểm tra phiên bản Docker
docker build -t myapp .   # Build image
docker images             # Liệt kê images
docker run -d -p 8080:80 myapp # Chạy container
docker ps                 # Liệt kê container đang chạy
docker stop <id>          # Dừng container
docker rm <id>            # Xoá container
```

→ Use Case Thực Tế:

- Triển khai Microservices
- CI/CD pipeline (Jenkins, GitHub Actions)
- Chạy database (MySQL, Postgres)
- Môi trường phát triển local
- Kubernetes (điều phối container)
- Triển khai trên Cloud (AWS, GCP, Azure)

→ Mẹo Sẵn Sàng Cho Production:

- ✓ Dùng official/base image (vd: node:18, openjdk:17)
- ✓ Giữ image nhỏ gọn (dùng multi-stage build)
- ✓ Dùng `.dockerignore` để bỏ file không cần thiết
- ✓ Không lưu secret trong image (dùng secrets manager)
- ✓ Scan image để tìm lỗ hổng (Trivy / Docker Scout)
- ✓ Dùng tag (latest, v1.0.0) và push lên registry (Docker Hub/ECR)

💡 **Pro Tip:**

"Docker giúp môi trường ứng dụng của bạn trở nên nhất quán, dễ di chuyển và sẵn sàng cho production."

Học ♥ Xây Dựng ♥ Cùng Nhau Phát Triển!

Chủ đề: Kiến Trúc Docker **Ngày: 15/08/2025**

"Tìm hiểu cách Docker hoạt động phía sau hậu trường"

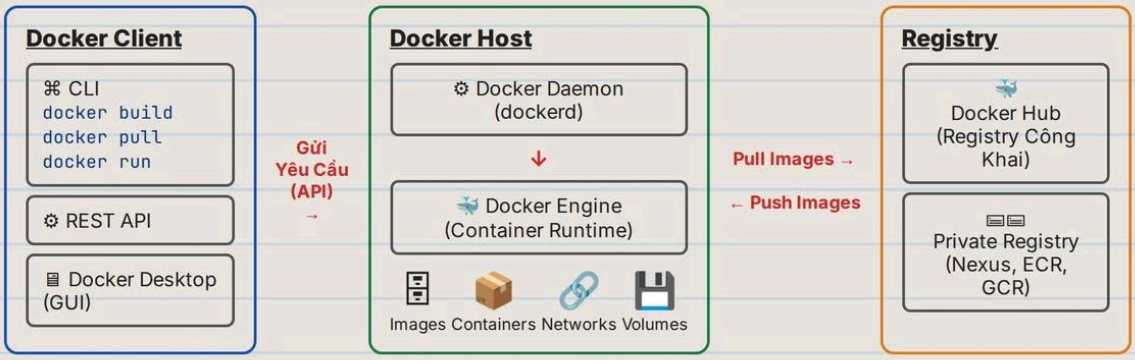
① Kiến Trúc Docker Là Gì?

→ Docker sử dụng kiến trúc **client-server**. Docker client gửi lệnh đến Docker daemon, daemon này quản lý container, image, network và volume trên Docker host. Image được pull từ / push lên một container registry (vd: Docker Hub).

Thành Phần Chính:

- ☑ **Docker Client** (CLI / API)
- ☑ **Docker Daemon** (dockerd)
- ☑ **Docker Host** (Máy chạy Docker)
- ☑ **Docker Engine** (Runtime cốt lõi)
- ☑ **Registry** (Docker Hub / Private)
- ☑ **Images** (Bản mẫu chỉ đọc)
- ☑ **Containers** (Instance đang chạy)

② Sơ Đồ Kiến Trúc Docker



③ Luồng Yêu Cầu (Từng Bước)

① User chạy một lệnh (vd. `docker run nginx`) trên Docker Client.

② Client gửi yêu cầu đến **Docker Daemon** (qua REST API).

③ Daemon kiểm tra xem image đã có sẵn ở local chưa.

④ Nếu chưa, daemon pull image từ Registry (Docker Hub/Private).

⑤ Daemon dùng Docker Engine để tạo và chạy container.

⑥ Container chạy trên Docker Host, dùng tài nguyên OS (namespaces, cgroups).

⑦ Output / logs được gửi trả về client.

💡 Điểm Chính:

→ Client & Daemon giao tiếp qua REST API (Unix socket / TCP).

→ Docker Engine dùng **containerd + runc** bên dưới.

→ Images chỉ đọc, containers có thể ghi.

→ Registry lưu trữ & phân phối Docker image.

④ Ví Von Đời Thực

→ **Client** → Bạn (Người đặt hàng)

→ **Daemon** → Bồi bàn (Xử lý yêu cầu)

→ **Engine** → Bếp (Thực hiện nấu ăn)

→ **Registry** → Siêu thị (Lưu nguyên liệu/image)

→ **Container** → Món ăn được phục vụ (Ứng dụng đang chạy)

⑤ Vì Sao Có Kiến Trúc Này?

- ✓ Tách biệt các mối quan tâm (Separation of concerns)
- ✓ Môi trường an toàn & cô lập
- ✓ Dễ dàng phân phối image
- ✓ Chạy trên mọi OS / Cloud
- ✓ Có khả năng mở rộng & sẵn sàng cho production

🚀 **Build · Ship · Scale với Docker!** 🐳

🚀 Học Hôm Nay... Xây Dựng Ngày Mai...

Cài Đặt & Kiến Thức Cơ Bản Docker

Cùng cài đặt Docker và chạy container đầu tiên của bạn 🙂

① Docker Là Gì?

→ Docker là nền tảng mã nguồn mở để build, ship và run ứng dụng trong **container**.

→ Container là môi trường **nhẹ**, **dễ di chuyển** và **cô lập** cho ứng dụng của bạn.

→ Docker giúp môi trường nhất quán từ development đến production.

Tại Sao Dùng Docker?

- ✓ Lightweight (Nhẹ)
- ✓ Portable (Di động)
- ✓ Consistent (Nhất quán)
- ✓ Faster Deployment (Triển khai nhanh)
- ✓ Works on any OS (Chạy trên mọi hệ điều hành)

② Cài Đặt Docker

🐧 Trên Ubuntu (Linux)

```
$ sudo apt update
$ sudo apt install -y docker.io
$ sudo systemctl start docker
$ sudo systemctl enable docker
$ sudo usermod -aG docker $USER
```

→ Đăng xuất và đăng nhập lại để áp dụng thay đổi group.

🪟 Trên Windows

1. Tải Docker Desktop từ → https://www.docker.com
2. Cài đặt và làm theo hướng dẫn
3. Khởi động Docker Desktop
4. Kiểm tra bằng `docker version`

③ Kiểm Tra Phiên Bản Docker

```
$ docker version
```

→ Hiển thị phiên bản client & server.

④ Thông Tin Docker

```
$ docker info
```

→ Hiển thị thông tin hệ thống, storage, container, v.v.

⑤ Trợ Giúp Docker

```
$ docker --help
```

→ Hiển thị tất cả các lệnh docker khả dụng.

⑥ Chạy Container Đầu Tiên

```
$ docker run hello-world
```

→ Docker sẽ pull image từ Docker Hub

→ Chạy container

→ Hiển thị thông báo thành công 🙂

💡 Pro Tip:

- ✓ Dùng `docker --help` để khám phá thêm
- ✓ Dùng `docker ps` để liệt kê container đang chạy
- ✓ Dùng `docker images` để liệt kê image đã tải

⇢ Từng Bước Nhỏ... Hành Trình DevOps Lớn! 🚀 ⇠

Ngày: 12 Aug 2025

Docker Containers

Vòng Đời Container: Create → Start → Stop → Restart → Remove
Exec | Logs | Inspect

→ Container là môi trường runtime **nhẹ**, **dễ di chuyển** và **cô lập** chạy một ứng dụng cùng toàn bộ dependency.

Create → Start → Run → Stop → Restart → Remove

① Tạo Container 🧊

```
docker create --name myapp nginx:latest
```

→ Tạo container (ở trạng thái dừng)
→ Không tự khởi động

② Khởi Động Container ▷

```
docker start myapp
```

→ Khởi động một container đã tồn tại

③ Chạy Container (create + start) ▷

```
docker run -d --name myapp -p 8080:80
nginx:latest
```

→ Tạo và khởi động container ở chế độ nền
→ -p → port mapping (host:container)

④ Dừng Container ■

```
docker stop myapp
```

→ Dừng container đang chạy một cách an toàn

⑤ Khởi Động Lại Container ⟳

```
docker restart myapp
```

→ Dừng rồi khởi động lại container

⑥ Xoá Container 🗑

```
docker rm myapp
```

→ Xoá container đã dừng
→ Dùng -f để ép xoá (container đang chạy)

```
docker rm -f myapp
```

⑦ Exec Vào Container 💻

```
docker exec -it myapp /bin/bash
```

→ Truy cập shell của container đang chạy
→ Dùng /bin/sh nếu không có bash

⑧ Xem Log Container 📄

```
docker logs myapp
```

→ Hiển thị log container
→ Dùng -f để theo dõi log real-time

```
docker logs -f myapp
```

⑨ Inspect Container 🔎

```
docker inspect myapp
```

→ Hiển thị thông tin chi tiết dạng JSON
→ Hữu ích để debug (IP, mounts, config...)

💡 Lệnh Hữu Ích

`docker ps` → Liệt kê container đang chạy
`docker ps -a` → Liệt kê tất cả container
`docker images` → Liệt kê image local
`docker rm $(docker ps -aq)` → Xoá tất cả container đã dừng
`docker system prune -f` → Dọn dẹp tài nguyên không dùng

⭐ Mẹo Hay (Sẵn Sàng Production)

- ✓ Đặt tên container có ý nghĩa
- ✓ Dùng -d để chạy nền
- ✓ Kiểm tra log để debug
- ✓ Dùng `docker inspect` để xem chi tiết
- ✓ Dọn dẹp container & image không dùng
- ✓ Dùng `docker compose` cho ứng dụng nhiều container

Build · Automate · Scale · Repeat 🚀

Ngày: 15 Aug 2025

Docker Images

Build Once → Run Anywhere

1) Docker Image Là Gì?

- → Một bản mẫu **chỉ đọc** dùng để tạo container.
- → Chứa source code ứng dụng, dependency, runtime, config, v.v.
- → Bất biến (không thay đổi được).
- → Được build theo layer (Union File System).
- → Được định nghĩa bằng Dockerfile.

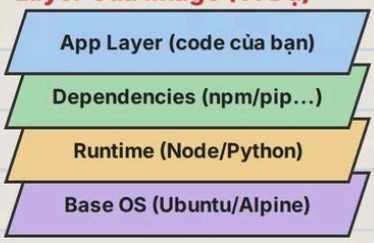
💡 Hãy hình dung như sau:

Image giống như **"bản thiết kế"** của ứng dụng bạn. Container là một instance đang chạy của bản thiết kế đó.

2) Các Layer Của Image

- → Image được tạo từ nhiều layer.
- → Mỗi chỉ thị trong Dockerfile tạo một layer.
- → Layer được cache, nên build nhanh hơn.
- → Layer chỉ đọc và được chia sẻ giữa các image (tiết kiệm dung lượng).

Layer Của Image (Ví Dụ)



3) Image ID

- → Định danh duy nhất cho một image.
- → Ví dụ: `sha256:1a2b3c4d5e6f...`
- → Thường chỉ hiện 12 ký tự đầu.
- → Cùng nội dung image sẽ có cùng ID.

```
IMAGE ID (Đầy đủ): sha256:1a2b3c4d5e6f7890...
IMAGE ID (Rút gọn): 1a2b3c4d5e6f
```

4) Tags

- → Tên thân thiện cho một image.
- → Định dạng: `<name>:<tag>`
- → Tag mặc định là `latest` (nếu không chỉ định).
- → Giúp quản lý các phiên bản khác nhau.

```
nginx:latest  → phiên bản mới nhất
nginx:1.25    → phiên bản cụ thể
myapp:v1      → tag tuỳ chỉnh
```

5) Tìm Kiếm Image 🔍

Tìm image trên Docker Hub

```
docker search nginx
```

→ Hiển thị các image có sẵn
→ Dùng --limit để giới hạn kết quả

```
docker search nginx --limit 5
```

6) Pull Image ⬇

Tải image từ registry

```
docker pull nginx:latest
```

→ Tải image về máy local
→ Chỉ định tag để lấy đúng phiên bản

7) Liệt Kê Image 📋

Hiển thị tất cả image local

```
docker images
```

→ Hiển thị REPOSITORY, TAG, IMAGE ID, SIZE
→ Dùng -a để hiện tất cả

```
docker images -a
```

8) Xoá Image 🗑

Xoá image khỏi máy local

```
docker rmi nginx:latest
```

→ Dùng tên hoặc image ID
→ Dùng -f để ép xoá

```
docker rmi -f 1a2b3c4d5e6f
```

⭐ Mẹo Hay

✓ Dùng tag cụ thể (tránh latest ở prod)
✓ Giữ image nhỏ gọn (dùng alpine)
✓ Dọn dẹp image không dùng thường xuyên

```
docker image prune -a
```

✓ Dùng multi-stage build cho image nhỏ hơn
✓ Scan image tìm lỗ hổng (Trivy/Docker Scout)

🚀 Use Case Thực Tế

1. Tìm base image
2. Pull phiên bản cần dùng
3. Build image tuỳ chỉnh
4. Gắn tag phiên bản
5. Push lên Docker Hub / ECR
6. Dùng trong pipeline CI/CD
7. Chạy như container trong Dev/Prod

♥ **Tiếp Tục Học, Tiếp Tục Phát Triển!**

Ghi Chú DevOps 🚀

Ngày: 15 Aug 2025

Dockerfile — Kiến Thức Nền Tảng

Dockerfile là một file văn bản chứa các chỉ thị để build một Docker image. Docker đọc file này và tạo image từng layer một.

🎯 Các Chỉ Thị Chính

1 FROM → **Thiết lập base image** cho container.
- Chỉ thị đầu tiên trong Dockerfile
- Có thể dùng image chính thức hoặc tự tạo

```
FROM node:20-alpine
```

2 RUN → **Thực thi một lệnh** trong quá trình build image.
- Dùng để cài package / công cụ
- Tạo một layer mới

```
RUN apk add --no-cache
    curl git
```

3 COPY → **Copy file/thư mục** từ máy local vào image.
- Dùng cho source code ứng dụng

```
COPY . /app
```

4 ADD → Tương tự **COPY**, nhưng có thêm tính năng.
- Có thể tải file từ URL
- Tự động giải nén file .tar

```
ADD https://example.com/
    app.tar.gz /app
```

5 WORKDIR → **Thiết lập thư mục làm việc** bên trong container.
- Tự tạo thư mục nếu chưa có
- Mọi lệnh sau đó chạy tại đây

```
WORKDIR /app
```

6 ENV → **Thiết lập biến môi trường.**
- Có sẵn trong cả lúc build & runtime

```
ENV NODE_ENV=production
ENV PORT=3000
```

7 EXPOSE → **Khai báo port** mà container sẽ lắng nghe.
- Không tự publish port (dùng -p lúc runtime)
- Chỉ mang tính tài liệu

```
EXPOSE 3000
```

8 CMD → **Lệnh mặc định** chạy khi container khởi động.
- Có thể bị override lúc runtime
- Nên dùng exec form

```
CMD ["node", "server.js"]
```

9 ENTRYPOINT → **Thiết lập executable chính** cho container.
- Không dễ bị override
- Dùng cho script khởi động cố định

```
ENTRYPOINT ["docker-
entrypoint.sh"]
CMD ["node", "server.js"]
```

✅ Thực Hành Tốt Nhất

- ✓ Dùng base image nhẹ (alpine)
- ✓ Gộp lệnh RUN để giảm layer
- ✓ Dùng file `.dockerignore`
- ✓ Ưu tiên COPY hơn ADD (trừ khi cần)
- ✓ Dùng multi-stage build để image nhỏ hơn
- ✓ Không hardcode secret trong Dockerfile

Dockerfile Mẫu

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
ENV NODE_ENV=production
EXPOSE 3000
CMD ["node", "server.js"]
```

⭐ Vì Sao Dùng Dockerfile?

- ✓ Build có thể tái lập
- ✓ Được quản lý phiên bản
- ✓ Di động trên mọi môi trường
- ✓ Thiết lập nhất quán
- ✓ Dễ mở rộng & triển khai

Build Một Lần, Chạy Ở Mọi Nơi! 🚀

Tiếp Tục Học. Tiếp Tục Phát Triển ♥

Ngày: 15-Aug-2025

Build & Tối Ưu Image Docker

Build nhanh hơn • Image nhỏ gọn hơn • Sẵn sàng Production 🚀

① Docker Build (Cơ Bản)

Build một image từ Dockerfile

```
docker build -t myapp:1.0 .
```

→ -t : tên tag

→ . : build context (thư mục hiện tại)

→ Đọc chỉ thị từ Dockerfile

→ Tạo ra một image mới

② Image Tags

Tag giúp đánh phiên bản & quản lý image

```
docker build -t myapp:1.0 .
docker build -t myapp:latest .
docker tag myapp:1.0 myrepo/myapp:1.0
```

✓ Dùng tag có ý nghĩa (v1.0, v2.1)

✓ Tránh chỉ dùng `latest` ở production

✓ Dùng semantic versioning

③ Build Context

File & thư mục gửi đến Docker daemon

→ Chỉ file trong build context được gửi đi

→ Dùng .dockerignore để loại trừ file

→ Giữ context nhỏ để build nhanh hơn

```
.dockerignore (ví dụ)
node_modules/
.git/
*.log
dist/
```

④ Build Cache

Docker cache layer để build nhanh hơn

→ Tái sử dụng layer không thay đổi

→ Sắp xếp chỉ thị (ít thay đổi trước)

→ Dùng COPY cụ thể thay vì copy hết

💡 Mẹo:
✓ COPY package*.json trước
✓ RUN npm install (cache nếu không đổi)
✓ Sau đó mới copy phần code còn lại

⑤ Multi-Stage Builds

Dùng nhiều stage để giảm dung lượng image cuối cùng

```
# Stage 1: Build
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Stage 2: Runtime
FROM nginx:alpine
COPY --from=builder /app/dist
/usr/share/nginx/html
```

✓ Image cuối nhỏ gọn hơn

✓ Không có công cụ build ở runtime

✓ Bảo mật hơn

✓ Sẵn sàng cho production

⑥ BuildKit (Nâng Cao)

Trình build thế hệ mới (nhanh hơn + cache tốt hơn)

```
# Bật BuildKit
export DOCKER_BUILDKIT=1
docker build -t myapp:1.0 .
```

Tính Năng:

✓ Build nhanh hơn

✓ Cache tốt hơn

✓ Chạy song song

✓ Hỗ trợ --secret, --mount

💡 Dùng buildx cho multi-platform build

```
docker buildx build \
  --platform
linux/amd64 .
```

🎯 Thực Hành Tốt Nhất (Production Ready)

✓ Dùng .dockerignore

✓ Giữ base image nhỏ (alpine/distroless)

✓ Dùng multi-stage build

✓ Tận dụng build cache

✓ Dùng tag cụ thể (tránh latest)

✓ Bật BuildKit / buildx

✓ Scan image tìm lỗ hổng

🎯 Image Nhỏ Hơn = Deploy Nhanh Hơn = Production Tốt Hơn!

Ngày: 15-08-2025

Dockerfile — Thực Hành Production

Tối ưu layer, .dockerignore, non-root user, image nhỏ gọn, bảo mật.

① Dùng Base Image Nhỏ & Chính Thức

- ✓ Dùng image chính thức (Docker Hub)
- ✓ Ưu tiên bản slim/alpine
- ✓ Tránh tag latest (dùng version cụ thể)

```
FROM eclipse-temurin:17-jre-alpine
```

↳ Nhỏ gọn, an toàn & ổn định

② Tối Ưu Layer (Cache Tốt Hơn)

- ✓ Copy file đúng thứ tự
- ✓ Tận dụng layer caching
- ✓ Gộp các lệnh RUN
- ✓ Tránh layer không cần thiết

```
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src/ ./src
RUN mvn package -DskipTests
```

③ Dùng .dockerignore

- ✓ Loại bỏ file không cần thiết
- ✓ Giảm dung lượng build context
- ✓ Build nhanh hơn & image nhỏ hơn

```
.dockerignore (ví dụ)
target/
.git/
node_modules/
*.log
.env
Dockerfile
```

④ Chạy Với Non-Root User

- ✓ Không bao giờ chạy với root
- ✓ Tạo user riêng biệt
- ✓ Tăng cường bảo mật container

```
RUN addgroup -S appgroup && \
    adduser -S appuser -G
appgroup

USER appuser
WORKDIR /app
```

⑤ Giữ Image Nhỏ Gọn

- ✓ Dùng multi-stage build
- ✓ Chỉ cài dependency cần thiết
- ✓ Xoá cache/file tạm
- ✓ Dùng image alpine/slim

```
# Stage 1 - Build
FROM maven:3.9-eclipse-
temurin-17 AS builder
RUN mvn package -DskipTests

# Stage 2 - Runtime
FROM eclipse-temurin:17-jre-
alpine
COPY --from=builder
/app/target/app.jar
/app/app.jar
```

⑥ Thực Hành Bảo Mật

- ✓ Dùng base image tối giản
- ✓ Chạy với non-root user
- ✓ Scan image tìm lỗ hổng
- ✓ Không lưu secret trong image
- ✓ Dùng read-only filesystem (nếu được)
- ✓ Cập nhật dependency thường xuyên

```
Scan Image:
docker scan myapp:latest
hoặc
trivy image myapp:latest
```

⭐ Dockerfile Production Ready (Ví Dụ)

```
# Dùng base image nhỏ, chính thức
FROM eclipse-temurin:17-jre-alpine

# Tạo user non-root
RUN addgroup -S appgroup && adduser -S
appuser -G appgroup

# Thiết lập thư mục làm việc
WORKDIR /app

# Chỉ copy file cần thiết
COPY target/app.jar app.jar

# Đổi quyền sở hữu
RUN chown -R appuser:appgroup /app

# Chuyển sang user non-root
USER appuser

# Chạy ứng dụng
ENTRYPOINT ["java", "-jar", "app.jar"]
```

💡 Mẹo Nhanh

→ Dùng tag phiên bản cụ thể (vd: 17.0.9)

→ Giữ Dockerfile đơn giản & sạch sẽ

→ Dùng multi-stage build

→ Xoá cache (apk, apt) sau khi cài

→ Test image ở local trước khi push

→ Tích hợp image scanning vào CI/CD

Kết Quả: Nhỏ Gọn ✓ Nhanh ✓ An Toàn ✓
Image Sẵn Sàng Cho Production ✓

Theo Dõi Thêm
Ghi Chú DevOps! ♥

Build Thông Minh 🚀 Ship An Toàn 🚀 Đi Đến Production 🚀

Docker Networking

Kết Nối Container • Giao Tiếp • Xây Dựng Ứng Dụng Có Thể Mở Rộng

① Vì Sao Cần Docker Networking?

- ✓ Các container cần giao tiếp với nhau
- ✓ Cô lập ứng dụng để bảo mật
- ✓ Cho phép giao tiếp với thế giới bên ngoài
- ✓ Hỗ trợ kiến trúc microservices có thể mở rộng



② Các Loại Docker Network

Bridge (Mặc định)	Host	None	Custom Network
• Network mặc định • Cô lập trên một host • Dùng NAT • Container giao tiếp được với nhau	• Dùng chung network namespace với host • Không NAT • Hiệu năng nhanh hơn • Ít cô lập hơn	• Không có network • Cô lập hoàn toàn • Không giao tiếp ngoài/trong • Dùng cho bảo mật cao	• Bridge do người dùng định nghĩa • Kiểm soát tốt hơn • Có DNS tích hợp • Khuyên dùng cho microservices
docker0 (bridge) ↓ Host	Host Network ↓ Dùng IP của host	✗ No Network	my-custom-net

③ Tạo & Dùng Network (Lệnh)

```
# Tạo custom network
docker network create mynet

# Chạy container trong cùng network
docker run -d --name app1 --network mynet nginx
docker run -d --name app2 --network mynet nginx

# Liệt kê network
docker network ls

# Inspect network
docker network inspect mynet
```

④ Giao Tiếp Container-to-Container

- ✓ Dùng tên container (DNS) để kết nối
- ✓ Không cần dùng địa chỉ IP
- ✓ Hoạt động sẵn trong custom network

```
# bên trong container app1
curl http://app2:80   # dùng tên container
```

⑤ Docker DNS (Service Discovery)

- ✓ Docker cung cấp DNS tích hợp cho custom network
- ✓ Container phân giải tên nhau tự động
- ✓ Không cần chỉnh /etc/hosts

```
Ví dụ phân giải DNS:
ping app2   # hoạt động nếu cùng network
```

⑥ Thực Hành Tốt Nhất

- ✓ Dùng custom network cho production
- ✓ Tránh --network host ở production (rủi ro bảo mật)
- ✓ Dùng tên container thay vì IP
- ✓ Phân vùng network cho từng service
- ✓ Dùng Docker Compose cho app nhiều container

💡 **Mẹo Hay:** Dùng custom network + Docker Compose + tên service để giao tiếp giữa container sạch sẽ, có thể mở rộng và an toàn!

Tiếp Tục Học · Tiếp Tục Xây Dựng · Cùng Nhau ♥

🚀 Học Hôm Nay... Xây Dựng Ngày Mai...

Docker Volumes & Storage

Giữ Dữ Liệu An Toàn, Bền Vững và Dễ Di Chuyển 🗄

① Vì Sao Storage Quan Trọng?

- Container mang tính tạm thời (dữ liệu mất khi container dừng).
- Dùng Volumes / Bind Mounts / tmpfs cho lưu trữ bền vững và tin cậy.
- Tách biệt code ứng dụng khỏi dữ liệu.
- Backup, restore và chia sẻ dễ dàng giữa các container.

② Các Loại Docker Storage

🗄 Volumes (Docker quản lý)
- Tốt nhất cho dữ liệu bền vững
- Lưu trong khu vực lưu trữ của Docker

📁 Bind Mounts (Host Path)
- Map thư mục host vào container
- Tốt cho file dev / config

💾 tmpfs (In-Memory)
- Lưu trong RAM (không ghi ổ đĩa)
- Nhanh & tạm thời (không bền vững)

③ Docker Volumes (Khuyên Dùng)

Tạo một volume:

```
docker volume create mydata
```

Chạy container với volume:

```
docker run -d \
  --name myapp \
  -v mydata:/app/data \
  nginx
```

④ Bind Mounts (Thư Mục Host)

Chạy với bind mount:

```
docker run -d \
  --name myapp \
  -v /host/path:/app/data \
  nginx
```

- ✓ Thay đổi trên host ↔ phản ánh trong container
- ✓ Hữu ích cho phát triển & file config
- ✓ Bạn kiểm soát chính xác đường dẫn host

⑤ tmpfs (Lưu Trong Bộ Nhớ)

Chạy với tmpfs:

```
docker run -d \
  --name myapp \
  --tmpfs /app/cache \
  nginx
```

- ✓ Lưu trong bộ nhớ (không ổ đĩa)
- ✓ Hiệu năng nhanh
- ✓ Mất dữ liệu khi container dừng
- ✓ Dùng cho cache / file tạm / dữ liệu nhạy cảm

⑥ Backup & Restore Volume

Backup một volume:

```
docker run --rm -v mydata:/data \
  -v $(pwd):/backup \
  alpine tar czf /backup/mydata_backup.tar.gz -C /data
.
```

Restore một volume:

```
docker run --rm -v mydata:/data \
  -v $(pwd):/backup \
  alpine tar xzf /backup/mydata_backup.tar.gz -C /data
```

⑦ Thực Hành Tốt Nhất

- ✓ Dùng named volume cho dữ liệu bền vững
- ✓ Giữ code ứng dụng & dữ liệu tách biệt
- ✓ Backup định kỳ các volume quan trọng
- ✓ Dùng tmpfs cho dữ liệu nhạy cảm/tạm thời
- ✓ Thiết lập đúng quyền (UID/GID)
- ✓ Dùng volume label để dễ quản lý

💡 Mẹo Hay:

- ✓ Liệt kê volume: `docker volume ls`
- ✓ Inspect volume: `docker volume inspect <name>`
- ✓ Xoá volume: `docker volume rm <name>`
- ✓ Kiểm tra dung lượng: `docker system df -v`

Build 🚀 Automate 🚀 Scale 🚀 Grow

Docker Compose

Chạy Ứng Dụng Nhiều Container Chỉ Với Một Lệnh

→ Docker Compose là công cụ để định nghĩa và chạy ứng dụng Docker nhiều container bằng một file YAML duy nhất (`compose.yaml`). Nó giúp đơn giản hoá việc quản lý **services**, **networks**, **volumes** và **biến môi trường** cho ứng dụng của bạn.

(1) compose.yaml

- ✓ File YAML để định nghĩa service
- ✓ Giữ hạ tầng dưới dạng code
- ✓ Khuyên dùng version 3.x
- ✓ Dễ chia sẻ & bảo trì

(2) Cấu Trúc Cơ Bản

```
version: "3.9"
services:
  app:
    image: myapp:latest
    ports:
      - "8080:8080"
```

(3) Services

- ✓ Mỗi container là một service
- ✓ Định nghĩa image hoặc build context
- ✓ Cấu hình port, env, volume, v.v.
- ✓ Nhiều service có thể chạy cùng nhau

(4) Networks

- ✓ Services giao tiếp qua network
- ✓ Tự động tạo network mặc định
- ✓ Bạn cũng có thể định nghĩa network tuỳ chỉnh

(5) Volumes

- ✓ Lưu dữ liệu bền vững bên ngoài container
- ✓ Dùng named hoặc bind volume
- ✓ Hữu ích cho database, config, log

(6) Biến Môi Trường

- ✓ Dùng `environment:` hoặc `env_file:`
- ✓ Lưu config như DB URL, API key
- ✓ Giữ secret ngoài image

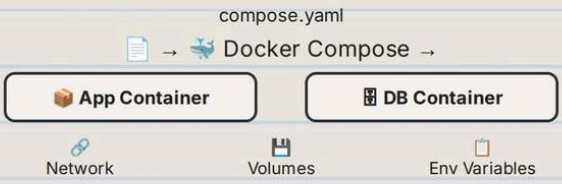
(7) Dependencies

- ✓ Dùng `depends_on` để kiểm soát thứ tự khởi động
- ✓ Không đợi service "sẵn sàng" (chỉ đợi thứ tự start)
- ✓ Muốn kiểm tra readiness thật sự, dùng `healthcheck`

★ Ví Dụ Đầy Đủ (compose.yaml)

```
version: "3.9"
services:
  app:
    build: ./app
    container_name: myapp
    ports:
      - "8080:8080"
    environment:
      - DB_URL=postgres://db:5432/appdb
      - APP_ENV=production
    depends_on:
      - db
    networks:
      - app-network
    volumes:
      - app-data:/app/data
  db:
    image: postgres:15
    container_name: postgres
    environment:
      - POSTGRES_DB=appdb
      - POSTGRES_USER=admin
      - POSTGRES_PASSWORD=admin123
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - app-network
networks:
  app-network:
    driver: bridge
volumes:
  app-data:
  db-data:
```

★ Hoạt Động Như Thế Nào?



★ Lệnh Hữu Ích

- → `docker compose up -d` — Chạy nền
- → `docker compose ps` — Kiểm tra service đang chạy
- → `docker compose logs -f` — Xem log
- → `docker compose down` — Dừng & xoá
- → `docker compose build` — Rebuild image

★ Thực Hành Tốt Nhất

- ✓ Dùng file .env cho giá trị nhạy cảm
- ✓ Dùng named volume để lưu bền vững
- ✓ Thêm healthcheck cho service quan trọng
- ✓ Dùng version image cụ thể (không dùng latest)
- ✓ Đưa compose.yaml vào version control (Git)

🚀 **Đơn Giản Hoá · Tự Động Hoá · Ship Nhanh Hơn!**

Docker Environment & Secrets

ENV, .env, Docker secrets, quản lý cấu hình, xử lý dữ liệu nhạy cảm

1) Biến Môi Trường Là Gì?

→ Cặp key-value dùng để truyền cấu hình vào container.

→ Giúp tách biệt config khỏi code.

→ Hữu ích cho các môi trường khác nhau (dev/staging/prod).

2) Dùng ENV Trong Dockerfile

```
FROM node:18
ENV APP_NAME=myapp
ENV PORT=3000
ENV LOG_LEVEL=info
CMD ["node", "app.js"]
```

3) Dùng File .env Với Docker

→ Tạo file .env (ở thư mục gốc dự án)

```
DB_HOST=postgres
DB_USER=appuser
DB_PASSWORD=strongpassword
APP_ENV=production
```

→ Chạy container với file .env

```
docker run --env-file .env myapp
```

4) Dùng Biến Môi Trường Với Docker Compose

```
version: '3.8'
services:
  app:
    image: myapp:latest
    env_file:
      - .env
    environment:
      - PORT=3000
      - LOG_LEVEL=info
```

5) Docker Secrets (Swarm Mode)

→ Dùng cho dữ liệu nhạy cảm (mật khẩu, API key).

→ Được lưu trữ an toàn trong Docker (đã mã hoá).

→ Được mount như file bên trong container.

```
# Tạo một secret
echo "mysecretpassword" | docker secret
create db_password -
```

```
# Dùng trong service (docker-compose.yml)
secrets:
  - db_password
services:
  app:
    image: myapp
    secrets:
      - db_password
```

Bên trong container:
Secret có sẵn tại:
/run/secrets/db_password ✓

6) Thực Hành Tốt Nhất

✓ Không bao giờ hardcode dữ liệu nhạy cảm

✓ Dùng file .env cho config không nhạy cảm

✓ Dùng Docker secrets cho mật khẩu/key

✓ Dùng external secret manager (AWS Secrets Manager, HashiCorp Vault)

✓ Dùng config khác nhau cho dev/staging/prod

✓ Giới hạn quyền truy cập phù hợp

✓ Xoay vòng (rotate) secret định kỳ

7) Quản Lý Cấu Hình

→ Dùng .env cho phát triển local

→ Dùng Docker secrets/Vault cho production

→ Dùng config file/mounted volume cho cấu hình phức tạp

→ Giữ config bên ngoài image (12 factor app)

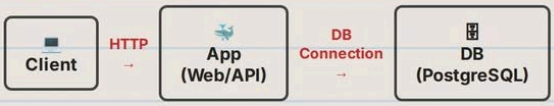
💡 **Điểm Cần Nhớ:**
Giữ code sạch, config tách biệt, và secret an toàn!
Dùng đúng công cụ cho đúng môi trường.

Build An Toàn · Deploy Tự Tin · Tiếp Tục Học!

Docker Compose Production Setup

Kiến trúc App + DB, healthchecks, secrets, restart policies, resource limits

1) Kiến Trúc (App + DB)



- ✓ Docker Compose
- ✓ Network cô lập
- ✓ Healthchecks
- ✓ Secrets
- ✓ Volume bền vững
- ✓ Credential an toàn
- ✓ Restart Policy
- ✓ Resource Limits

2) Cấu Trúc Dự Án

```
📁 docker-compose.yml
📁 .env (để ngoài git)
📁 secrets/
  └ db_password.txt
  └ app_secret.txt
📁 app/
  └ Dockerfile
  └ app code ...
```

Dùng .env cho giá trị không nhạy cảm. Dùng Docker secrets cho dữ liệu nhạy cảm (ở production).

3) Healthchecks

- ✓ Đảm bảo container thực sự sẵn sàng
- ✓ Compose đợi DB healthy
- ✓ Ngăn app lỗi lúc khởi động

```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U
$${POSTGRES_USER}"]
  interval: 10s
  timeout: 5s
  retries: 5
```

4) Restart Policies

- ✓ Giữ container chạy khi có lỗi
- ✓ Tốt nhất cho độ ổn định production

- `no` – Không restart
- `on-failure` – Chỉ restart khi lỗi
- `always` – Luôn restart
- `unless-stopped` – Restart trừ khi dừng thủ công **(Khuyên dùng)**

5) docker-compose.yml (Production Ready)

```
version: '3.9'
services:
  app:
    build: ./app
    container_name: myapp
    ports:
      - "8080:8080"
    env_file:
      - .env
    secrets:
      - app_secret
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "curl", "-f",
"http://localhost:8080/health"]
      interval: 10s
      timeout: 5s
      retries: 5
    restart: unless-stopped
    deploy:
      resources:
        limits:
          cpus: "0.50"
          memory: "512M"
        reservations:
          cpus: "0.25"
          memory: "256M"
  db:
    image: postgres:16-alpine
    container_name: mydb
    environment:
      POSTGRES_DB: myappdb
      POSTGRES_USER: myappuser
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    volumes:
      - db_data:/var/lib/postgresql/data
    secrets:
      - db_password
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U
$${POSTGRES_USER}"]
      interval: 10s
      timeout: 5s
      retries: 5
    restart: unless-stopped
secrets:
  db_password:
    file: ./secrets/db_password.txt
  app_secret:
    file: ./secrets/app_secret.txt
volumes:
  db_data:
```

6) Resource Limits (Vì Sao?)

- ✓ Ngăn một container chiếm hết tài nguyên hệ thống
- ✓ Cải thiện độ ổn định & hiệu năng
- ✓ Hỗ trợ môi trường nhiều service

> 💡 **Mẹo Cuối:**
> Docker Compose rất tốt cho các thiết lập production nhỏ đến vừa. Với quy mô lớn, hãy cân nhắc dùng công cụ điều phối như Kubernetes.

7) Thực Hành Tốt Nhất (Production)

- ✓ Dùng .env cho config (không phải secret)
- ✓ Dùng Docker secrets cho dữ liệu nhạy cảm
- ✓ Thêm healthcheck cho mọi service
- ✓ Đặt restart: unless-stopped
- ✓ Dùng resource limits
- ✓ Dùng named volume cho dữ liệu bền vững
- ✓ Giữ image tối giản (alpine/distroless)
- ✓ Dùng network riêng biệt (mặc định trong compose)
- ✓ Giám sát log (docker compose logs -f)

Build · Automate · Deploy · Grow ⟶

Docker Security 🔒

Build Image An Toàn · Chạy Container An Toàn Hơn · Tuân Thủ Nguyên Tắc Đặc Quyền Tối Thiểu

Container non-root, capabilities, filesystem chỉ đọc, scan image, secrets

1) Dùng Container Non-Root

→ Không bao giờ chạy container với root.

→ Dùng user non-root để tăng bảo mật.

→ Tạo user riêng trong Dockerfile.

```
FROM node:20-alpine
RUN addgroup -S appgroup && \
    adduser -S appuser -G appgroup
USER appuser
WORKDIR /app
COPY . .
CMD [ "node", "app.js" ]
```

↳ Chạy với user non-root ✓

2) Giới Hạn Linux Capabilities

→ Drop hết capability không cần thiết.

→ Chỉ thêm những gì thực sự cần.

```
docker run \
  --cap-drop ALL \
  --cap-add NET_BIND_SERVICE \
  myapp:latest
```

•• Dùng `--cap-drop ALL` để thiết lập an toàn nhất ✓

3) Dùng Filesystem Chỉ Đọc

→ Mount filesystem container ở chế độ read-only.

→ Chỉ cho phép ghi nơi thật sự cần.

```
docker run --read-only \
  --tmpfs /tmp \
  --tmpfs /var/run \
  myapp:latest
```

↳ Ngăn chặn ghi độc hại ✓

4) Scan Image Tìm Lỗ Hổng

→ Scan image trước khi dùng ở production.

→ Dùng công cụ uy tín (Trivy, Docker Scout).

```
# Dùng Trivy
trivy image myapp:latest

# Dùng Docker Scout
docker scout cves myapp:latest
```

✓ Tìm CVE ✓ Fix lỗi ✓ Dùng base image tối giản

5) Dùng Secrets (Đừng Hardcode)

→ Dùng Docker secrets cho dữ liệu nhạy cảm.

→ Không bao giờ lưu mật khẩu/API key trong image.

```
echo "mysecret" | docker secret create db_pass
-

# docker-compose.yml
services:
  app:
    image: myapp
    secrets:
      - db_pass
secrets:
  db_pass:
    external: true
```

Truy cập bên trong container: `/run/secrets/db_pass` ✓

6) Tuân Thủ Đặc Quyền Tối Thiểu

- ✓ Dùng user non-root
- ✓ Drop capability không cần thiết
- ✓ Dùng read-only filesystem
- ✓ Chỉ cho phép truy cập network cần thiết
- ✓ Dùng base image tối giản (alpine/distroless)
- ✓ Giới hạn bằng chính sách bảo mật (AppArmor/SELinux)
- ✓ Cập nhật image thường xuyên

7) Mẹo Bảo Mật Thêm

✅ Dùng base image chính thức & tin cậy

🔄 Cập nhật Docker & host OS

🚫 Không expose port không cần thiết

📄 Bật logging & giám sát

🔒 Dùng image signing (Docker Content Trust)

💡 Điểm Chính:

Bảo mật không phải là một tính năng, đó là **một thói quen!** Luôn tuân thủ đặc quyền tối thiểu, scan thường xuyên, và giữ secret an toàn.

Build An Toàn · Deploy Tự Tin · Tiếp Tục Học!

Quản Lý Tài Nguyên Docker

Kiểm soát & tối ưu CPU, Memory, Disk usage của container để đạt hiệu năng, độ ổn định và tối ưu chi phí tốt hơn.

① Giới Hạn & Đặt Trước CPU

- `--cpus` : Giới hạn CPU sử dụng (theo core)
- `--cpu-shares` : Trọng số tương đối (đặt trước)
- `--cpuset-cpus` : Gắn vào core CPU cụ thể

```
docker run -d \
  --name myapp \
  --cpus="1.5" \          # giới hạn 1.5 core
  --cpu-shares=512 \      # đặt trước (tương đối)
  --cpuset-cpus="0,1" \   # chỉ dùng core 0 & 1
  nginx:latest
```

💡 Mẹo:

- Dùng limit để tránh vấn đề "noisy neighbor".
- Dùng reservation để lập lịch công bằng.
- Trong Swarm/K8s, dùng `deploy.resources` để kiểm soát tốt hơn.

② Giới Hạn & Đặt Trước Memory

- `--memory` : Giới hạn cứng (vd: 512m, 1g)
- `--memory-reservation` : Giới hạn mềm (đặt trước)

```
docker run -d \
  --name myapp \
  --memory="512m" \                 # giới hạn cứng
  --memory-reservation="256m" \     # giới hạn mềm
  nginx:latest
```

③ OOM (Hết Bộ Nhớ)

- Nếu container vượt giới hạn memory → Linux kernel sẽ kill container (OOM Kill).
- Kiểm tra exit code: `137` (OOM)
- Ngăn chặn bằng cách đặt giới hạn memory hợp lý và giám sát usage.

```
docker inspect <container_id> |
  grep -i oom
```

④ Kiểm Tra Tài Nguyên Sử Dụng (Stats)

• CPU / Memory / Network / Block I/O theo thời gian thực

```
docker stats
docker stats <container_id>
```

CONTAINER	CPU %	MEM USAGE/LIMIT	NET I/O	BLOCK I/O
myapp	0.52%	120MiB / 512MiB	1.2MB / 1.1MB	3.5MB / 1.1MB
nginx	0.21%	45MiB / 256MiB	0.8MB / 0.6MB	1.2MB / 0.4MB

⑤ Dung Lượng Ổ Đĩa

- Kiểm tra dung lượng dùng bởi image, container, volume
- Giúp tìm ra thứ đang chiếm ổ đĩa

```
docker system df
docker system df -v
docker ps -s          # dung lượng theo container
```

⑥ Dọn Dẹp Container

• Xoá container đã dừng

```
docker container prune -f
```

• Xoá image không dùng

```
docker image prune -a -f
```

• Dọn dẹp toàn bộ (container, image, network, build cache)

```
docker system prune -a --volumes -f
```

✅ Thực Hành Tốt Nhất

- ✓ Luôn đặt giới hạn CPU & memory ở production.
- ✓ Giám sát bằng docker stats / Prometheus + Grafana.
- ✓ Dùng healthcheck để phát hiện container bị treo.
- ✓ Dọn dẹp tài nguyên không dùng thường xuyên.
- ✓ Dùng base image nhỏ hơn để tiết kiệm dung lượng.

📌 Ghi Nhớ:

- **Limits** = Mức tối đa cho phép
- **Reservations** = Mức tối thiểu đảm bảo
- **OOM Kill** = Exit code 137
- **Dọn dẹp** = Dung lượng & hiệu năng tốt hơn

Build · Monitor · Optimize · Grow 🚀

🚀 Học Hôm Nay... Xây Dựng Ngày Mai... *Ngày: 23/08/2025*

Docker Registry

Lưu Trữ, Quản Lý và Phân Phối Docker Images 🎯

① Docker Registry Là Gì?

- Docker Registry là một dịch vụ **lưu trữ** chứa các Docker image.
- Giúp chia sẻ image giữa các team, môi trường và triển khai.
- **Docker Hub** là một registry công khai.
- Chúng ta cũng có thể chạy một **private registry** (on-prem / nội bộ).

② Các Loại Registry

☁ **Docker Hub (Công Khai)**

- Registry chính thức công khai
- Miễn phí cho image công khai
- docker.io (mặc định)

🖴 **Private Registry**

- Tự host (vd: dùng registry:2)
- Harbor, Nexus, GitLab Registry
- Dùng cho image nội bộ/công ty

③ Đăng Nhập Registry

• Docker Hub

```
docker login
```

Nhập username & password Docker Hub

• Private Registry

```
docker login <registry-url>
```

Ví dụ: docker login myregistry.company.com

④ Gắn Tag Cho Image

```
docker tag <image-name>:<old-tag>
  <registry-url>/<image-name>:<new-tag>
```

Ví dụ: docker tag myapp:1.0 \
docker.io/yourusername/myapp:1.0

⑤ Push Image Lên Registry

```
docker push <registry-url>/<image-name>:<tag>
```

Ví dụ: docker push docker.io/yourusername/myapp:1.0

⑥ Pull Image Từ Registry

```
docker pull <registry-url>/<image-name>:<tag>
```

Ví dụ: docker pull docker.io/yourusername/myapp:1.0

⑦ Đánh Version Image (Thực Hành Tốt)

- ✓ Dùng tag có ý nghĩa (vd: v1.0.0, v1.1.0)
- ✓ Chỉ dùng latest cho phát triển
- ✓ Dùng semantic versioning (MAJOR.MINOR.PATCH)
- ✓ Gắn tag theo commit hash để dễ truy vết
- ✓ Tự động hoá việc gắn tag trong CI/CD pipeline

```
Ví dụ:
• myapp:1.0.0
• myapp:1.1.0
• myapp:latest (không khuyên dùng cho prod)
• myapp:abc123 (commit hash)
```

⑧ Chạy Private Registry (Đơn Giản)

① Chạy container registry

```
docker run -d -p 5000:5000 --name registry \
  registry:2
```

② Gắn tag & push

```
docker tag myapp:1.0 localhost:5000/myapp:1.0
docker push localhost:5000/myapp:1.0
```

③ Pull từ private registry

```
docker pull localhost:5000/myapp:1.0
```

💡 **Mẹo Production:**

✓ Dùng private registry cho image nhạy cảm
✓ Bật xác thực (TLS)
✓ Dùng image scanning (Trivy / Harbor)
✓ Thiết lập chính sách retention
✓ Dùng tag bất biến (immutable)
✓ Tích hợp với CI/CD (Jenkins / GitHub Actions)

Image Tốt Hơn... Triển Khai Mượt Hơn... Team Mạnh Hơn... 🚀

Docker Logging & Troubleshooting

Tìm lỗi nhanh hơn • Giữ container khoẻ mạnh • Chạy production ready ✅

① Docker Logs (Xem Log Container)

• Xem log theo thời gian thực

```
docker logs <container_name>
```

• Theo dõi log (live)

```
docker logs -f <container_name>
```

• Hiện 100 dòng cuối

```
docker logs --tail 100 <container_name>
```

• Từ thời điểm cụ thể

```
docker logs --since 1h <container_name>
```

② Logging Drivers

Docker hỗ trợ nhiều logging driver:

→ json-file	Driver mặc định (lưu log dạng file)
→ local	Hiệu năng tốt hơn, ít dung lượng hơn
→ syslog	Gửi log đến syslog server
→ journald	Dùng systemd journal (Linux)
→ awslogs	Gửi log đến AWS CloudWatch
→ fluentd	Gửi log đến Fluentd
→ gelf	Gửi log đến Graylog

Thiết lập logging driver lúc chạy container:

```
docker run --log-driver=awslogs \
  --log-opt awslogs-region=ap-south-1 \
  --log-opt awslogs-group=my-app  nginx
```

③ Debug Container

→ Kiểm tra trạng thái container

```
docker ps -a
```

→ Inspect container (chi tiết)

```
docker inspect <container_name>
```

→ Truy cập shell container

```
docker exec -it <container_name> sh
```

→ Kiểm tra tài nguyên sử dụng

```
docker stats
```

→ Kiểm tra tiến trình bên trong

```
docker exec -it <container_name> ps aux
```

→ Kiểm tra log + lỗi container

```
docker logs -f <container_name>
```

Inspect → Check → Fix → Repeat

④ Lỗi Liên Quan Đến Image

Không tìm thấy image	→ Kiểm tra tên & tag: `docker pull <image:tag>`
Sai phiên bản image	→ Dùng tag cụ thể (tránh latest): `docker pull nginx:1.25`
Image quá lớn	→ Dùng multi-stage build & .dockerignore
Image bị lỗi	→ Xoá & pull lại: `docker rmi <image>; docker pull <image>`
Build thất bại	→ Kiểm tra Dockerfile, context & log: `docker build --no-cache -t my-app .`

⑤ Lỗi Liên Quan Đến Network

→ Container không kết nối được internet
- Kiểm tra network mode (bridge/host)
- Kiểm tra DNS → `docker exec <c> nslookup google.com`
- Kiểm tra proxy / firewall rules

→ Giao tiếp container-to-container
- Dùng custom network: `docker network create my-network`
- Gắn container vào cùng network

→ Port không truy cập được
- Kiểm tra port mapping → `docker ps`
- Kiểm tra security group / firewall
- Kiểm tra app có lắng nghe ở `0.0.0.0` không

★ Lệnh Thường Dùng

`docker ps -a` → Liệt kê container `docker stats` → Tài nguyên

`docker logs -f <c>` → Theo dõi log `docker exec -it <c> sh` → Truy cập shell

`docker inspect <c>` → Chi tiết đầy đủ `docker system df` → Dung lượng

⑥ Lỗi Liên Quan Đến Disk / Storage

→ Kiểm tra dung lượng ổ đĩa

```
docker system df
```

→ Dọn dữ liệu không dùng

```
docker system prune -a
docker volume prune
```

→ Log chiếm quá nhiều dung lượng
- Dùng local logging driver
- Thiết lập log rotation

```
--log-opt max-size=10m
--log-opt max-file=3
```

Mẹo Production ✅

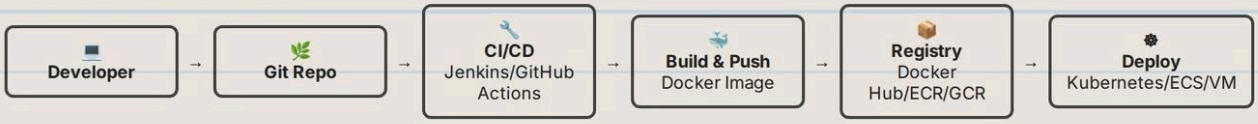
- ✓ Dùng structured logging (JSON)
- ✓ Gửi log đến CloudWatch / ELK / Loki
- ✓ Thiết lập log rotation để tránh đầy đĩa
- ✓ Giám sát sức khoẻ & tài nguyên container
- ✓ Dùng tag image cụ thể (tránh latest)
- ✓ Giữ image nhỏ gọn (multi-stage build)

Debug Thông Minh → Tiết Kiệm Thời Gian → Tiếp Tục Học 🚀

Production Docker + Interview Cheat Sheet

Kiến trúc production, thực hành tốt nhất, lỗi thường gặp, câu hỏi phỏng vấn Docker

1) Kiến Trúc Production



2) Dockerfile (Production Ready)

```
FROM eclipse-temurin:17-jre-alpine  # nhỏ & an toàn
WORKDIR /app
COPY target/app.jar app.jar

RUN addgroup -S app && adduser -S app -G app
USER app  # user non-root

EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

3) Thực Hành Tốt Nhất

- ✓ Dùng base image chính thức & tối giản
- ✓ Giữ image nhỏ (dùng .dockerignore)
- ✓ Không chạy container với root
- ✓ Dùng multi-stage build
- ✓ Scan image tìm lỗ hổng
- ✓ Dùng tag image cụ thể (tránh latest)
- ✓ Lưu secret bằng Secrets Manager / ENV
- ✓ Dùng health checks
- ✓ Đặt resource limits (CPU/Memory)
- ✓ Dùng read-only filesystem (khi có thể)
- ✓ Log ra stdout / stderr
- ✓ Tự động hoá build với CI/CD

4) Lệnh Docker Hữu Ích

```
$ docker build -t myapp:1.0 .          # Build image
$ docker run -d --name app -p 8080:8080 myapp:1.0
$ docker ps                            # Liệt kê container
$ docker logs -f app                   # Xem log
$ docker exec -it app sh               # Truy cập container
$ docker stats                         # Tài nguyên sử dụng
$ docker system prune -a               # Dọn dẹp
```

5) Lỗi Thường Gặp & Cách Khắc Phục

Vấn Đề	Cách Khắc Phục
• Container thoát ngay lập tức	→ Kiểm tra log (docker logs), entrypoint/CMD
• Image quá lớn	→ Dùng alpine, multi-stage build
• Port đã được dùng	→ Đổi port host (vd: 8081:8080)
• Permission denied	→ Dùng non-root user / sửa quyền file
• Hết bộ nhớ	→ Đặt memory limit / tối ưu ứng dụng
• Không pull được image	→ Kiểm tra xác thực registry / network
• App không truy cập được	→ Kiểm tra EXPOSE, port mapping, firewall

6) Docker Trong Production (Mở Rộng)

📦 Dùng Container Orchestration	→ Kubernetes / ECS
🗄 Dùng Private Registry	→ ECR / Harbor / Nexus
🔄 Dùng Rolling Updates	→ Zero downtime
❤ Thêm Health Checks	→ Giám sát sức khoẻ container
📋 Dùng Centralized Logging	→ ELK / CloudWatch / Loki
🔒 Dùng Image Scanning	→ Trivy / SonarQube / Snyk

7) Top Câu Hỏi Phỏng Vấn Docker

1. Docker container là gì và khác VM như thế nào?
2. Điều gì xảy ra khi bạn chạy 'docker run'?
3. Sự khác biệt giữa COPY và ADD?
4. Docker layer là gì? Chúng giúp ích ra sao?
5. Làm sao để giảm dung lượng Docker image?
6. Multi-stage build là gì?
7. Làm sao để lưu dữ liệu bền vững trong Docker?
8. Docker Compose là gì? Khi nào nên dùng?
9. Cách xử lý secret trong Docker?
10. Các thực hành bảo mật phổ biến trong Docker?

8) .dockerignore (Ví Dụ)

```
node_modules/
target/
.git/
*.log
Dockerfile
.env
```

Nhỏ Gọn Hơn
An Toàn Hơn
Nhanh Hơn
Sẵn Sàng Production!

💡 **Điểm Chính:**

"Dùng Docker đúng cách — Build image nhỏ gọn, an toàn và có khả năng mở rộng để thành công ở production!"

Theo Dõi | Thích | Tiếp Tục Học! ♥

Dự Án Tốt Nghiệp

Làm xong dự án này và tick hết checklist = bạn dùng Docker được ở mức đi làm thực tế.

🚀 Đề bài: "Task Board" chạy hoàn toàn bằng Docker

- **frontend**: React + Vite, multi-stage build → `nginx:alpine`, nginx proxy `/api` sang backend.
- **api**: Spring Boot, multi-stage (maven → `eclipse-temurin:21-jre-alpine`), chạy non-root, có endpoint `/actuator/health`.
- **db**: MySQL 8 với named volume, mật khẩu đọc từ secret file, healthcheck bằng `mysqladmin ping`.
- `compose.yaml`: network riêng, `depends_on` theo healthcheck, `restart: unless-stopped`, giới hạn memory, log rotation.
- GitHub Actions: build → scan (Trivy) → push image lên Docker Hub với tag `1.0.0` và git SHA.

Checklist tự đánh giá

- ✔ Clone về máy sạch, 1 lệnh là chạy.
- ✔ Không container nào chạy bằng root.
- ✔ Không có secret nào trong image (kiểm bằng `docker history`).
- ✔ Frontend image < 50 MB, API image < 250 MB.
- ✔ Sửa 1 dòng code → rebuild dùng cache cho phần dependency.
- ✔ `docker compose down` rồi `up` lại: dữ liệu vẫn còn.
- ✔ Tắt DB → API báo unhealthy rồi tự hồi phục khi DB lên lại.
- ✔ Không còn CVE mức CRITICAL khi scan.
- ✔ Image có tag semver, không deploy bằng `latest`.
- ✔ Giải thích được mọi dòng trong Dockerfile và compose.

Sau Docker thì học gì?

Kubernetes (Deployment, Service, Ingress — điều phối container ở quy mô lớn) → **CI/CD nâng cao** (GitHub Actions, môi trường preview) → **Container trên cloud** (AWS ECR + ECS/Fargate, hoặc GKE/AKS). Docker là nền móng cho cả ba.

💬 Lời cuối của mentor

Đừng đợi "học xong" mới dùng. Ngay tuần này, hãy Docker hoá một project nhỏ bạn đang làm. Mang Dockerfile đầu tiên của bạn đến cho mình review — đó là cách học nhanh nhất.

Ngày: 12 Aug 2025

Bảng Tra Cứu Lệnh Docker

Cheat Sheet

Các lệnh Docker hay dùng nhất | Ví dụ thực tế | Mẹo debug

Học • Luyện tập • Dùng trong dự án thực tế 🚀

① Images (Build, Pull, List, Remove) 📦

```
docker pull nginx                    # Pull image từ Docker Hub
docker images                        # Liệt kê tất cả image ở local
docker build -t myapp:1.0 .          # Build image
docker rmi myapp:1.0                 # Xoá image
docker image prune -f                # Xoá image không dùng
```

② Containers (Vòng đời, Dừng, Xoá) 📦

```
docker run -d --name web nginx       # Chạy container
docker ps                            # Liệt kê container đang chạy
docker ps -a                         # Liệt kê tất cả container
docker stop web                      # Dừng container
docker start web                     # Khởi động container
docker restart web                   # Khởi động lại container
docker rm web                        # Xoá container
docker rm -f web                     # Ép xoá
```

③ Exec & Logs 💻

```
docker exec -it web /bin/bash        # Mở shell
docker exec web ls -l /app           # Chạy lệnh
docker logs web                      # Xem log
docker logs -f web                   # Theo dõi log (live)
docker logs --tail 100 web           # 100 dòng cuối
```

④ Inspect & Info 🔍

```
docker inspect web                   # Thông tin chi tiết (JSON)
docker inspect nginx                 # Chi tiết image
docker port web                      # Hiển thị port mapping
docker top web                       # Tiến trình đang chạy
docker stats                         # Tài nguyên sử dụng live
```

⑤ Volumes (Lưu Dữ Liệu) 🗄

```
docker volume create mydata          # Tạo volume
docker volume ls                     # Liệt kê volume
docker run -d -v mydata:/app/data nginx
docker volume rm mydata              # Xoá volume
docker volume prune -f               # Xoá volume không dùng
```

⑥ Networks 🔗

```
docker network ls                    # Liệt kê network
docker network create mynet          # Tạo network
docker run -d --name web --network mynet nginx
docker network inspect mynet         # Chi tiết network
docker network rm mynet              # Xoá network
```

⑦ Debugging (Xử Lý Sự Cố) 🔧

```
docker ps -a                         # Kiểm tra container đã exit chưa
docker logs <container>              # Xem log
docker inspect <container>           # Tìm chi tiết lỗi
docker exec -it <container> sh       # Thử /bin/sh nếu không có bash
docker stats                         # Kiểm tra CPU / Memory
docker events                        # Sự kiện real-time
docker system df                     # Dung lượng ổ đĩa
```

⑧ Cleanup (Dọn Dẹp) 🧹

```
docker container prune -f            # Xoá container đã dừng
docker image prune -a                # Xoá image không dùng
docker volume prune -f               # Xoá volume không dùng
docker network prune -f              # Xoá network không dùng
docker system prune -a               # Dọn dẹp toàn bộ
```

💡 Mẹo Hay

- ✓ Đặt tên có ý nghĩa → `docker run --name myapp ...`
- ✓ Dùng tag → `myapp:1.0, myapp:latest`
- ✓ Kiểm tra log trước khi container lỗi
- ✓ Dùng `docker inspect` để debug sâu
- ✓ Dọn dẹp tài nguyên không dùng thường xuyên

⭐ Lỗi Thường Gặp & Cách Khắc Phục

Container tự thoát ngay	→ Kiểm tra log (`docker logs`)
Port đã được dùng	→ Đổi port host (vd: -p 8081:80)
Permission denied	→ Thêm sudo hoặc kiểm tra quyền volume
Hết dung lượng	→ Chạy `docker system prune -a`
App không truy cập được	→ Kiểm tra network & port mapping

Build · Run · Debug · Ship 🚀